SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5 **Total Marks:50**

Annexure A

SIMULATION TASK

Code of Conduct:

I _A. Midhuna (192521302) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Midhuna

QUESTION 1

Given the three-address code sequence:

t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y

Peephole Optimization:

Step 1: Algebraic Simplification

The statement t1 = x * 1 can be simplified using the algebraic identity:

x * 1 = x

Therefore:

t1 = x

The statement t3 = t2 - 0 can also be simplified using:

x - 0 = x

Therefore:

t3 = t2

Step 2: Constant Folding

The statement:

t4 = t3 * 0

can be simplified using:

x * 0 = 0

Therefore:

t4 = 0

Step 3: Conditional Branch Simplification

The original statement is:

if t4 != 0 goto L1

Since t4 = 0, the condition becomes:

if 0! = 0 goto L1

This condition is always false. Therefore, the conditional jump is removed.

Step 4: Dead Code Elimination

The unnecessary temporary assignments can be removed if their values are not used later. Copy propagation can also be applied.

Optimized Code:

t1 = x
t2 = x + y
t5 = y + y

Thus, the optimization removes multiplication by 1, subtraction of 0, multiplication by 0, the always-false conditional branch, and unnecessary temporary variables.

Conclusion:

Peephole optimization improves the code by replacing unnecessary operations with simpler equivalent operations. It reduces the number of instructions and improves execution efficiency.

QUESTION 2

Given Intermediate Code:

(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2

Leader Identification Rules:

The rules for identifying leaders are:

1. The first statement of the intermediate code is a leader.
2. The target of a conditional or unconditional jump is a leader.
3. The statement immediately following a conditional or unconditional jump is a leader.

Leader Statements:

Statement 1 is a leader because it is the first statement.

Statement 4 is a leader because it immediately follows the conditional jump at statement 3.

Statement 6 is a leader because it is the target of the conditional jump.

Statement 7 is a leader because it is the target of the unconditional jump.

Therefore:

Leaders = {1, 4, 6, 7}

Basic Blocks:

Basic Block B1:

(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6

Basic Block B2:

(4) z = y - d
(5) goto 7

Basic Block B3:

(6) z = y + d

Basic Block B4:

(7) w = z * 2

Control Flow Graph:

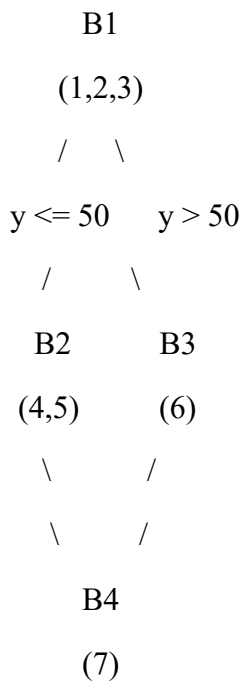
B1 → B2 when y <= 50

B1 → B3 when y > 50

B2 → B4

B3 → B4

Text Representation of CFG:



Conclusion:

The intermediate code is divided into four basic blocks. The control flow graph clearly represents the possible flow of execution between these blocks.

QUESTION 3

Given Basic Block:

p = a + b
q = a + b
r = p * q
s = a + b
t = r + s

DAG Construction:

The expression a + b occurs three times:

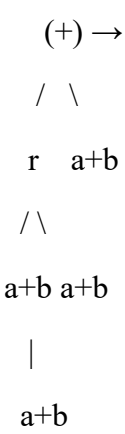
p = a + b
q = a + b
s = a + b

Instead of creating three separate nodes, the DAG creates one common node for a + b.

DAG Nodes:

N1 = a
N2 = b
N3 = N1 + N2 → p, q, s
N4 = N3 * N3 → r
N5 = N4 + N3 → t

DAG Representation:



Common Sub-Expression:

The common sub-expression is:

a + b

It occurs three times in the original code. The DAG identifies that the same expression can be calculated once and reused.

Original Code:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

Optimized Three-Address Code:

```
t1 = a + b
r = t1 * t1
t = r + t1
```

Optimized Target Code:

```
LOAD R0, a
ADD R0, b
MOV t1, R0
MUL R0, R0
MOV r, R0
ADD R0, t1
MOV t, R0
```

Register Allocation:

1. R0 is loaded with a.
2. b is added to R0, producing a + b.
3. The common value is stored as t1.
4. R0 is reused to calculate t1 * t1.
5. The multiplication result is stored in r.
6. The previously stored common value t1 is added to r.
7. The final result is stored in t.

Conclusion:

The DAG eliminates repeated computation of the common sub-expression a + b. This reduces the number of instructions and improves the efficiency of the generated code.

QUESTION 4

Given Three-Address Code:

t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4

Target Machine:

Registers: R0 and R1

Initial Register Descriptor:

R0 = Empty
R1 = Empty

Initial Address Descriptor:

a → Memory
b → Memory
c → Memory
d → Memory
e → Memory
f → Memory
t1 → Memory
t2 → Memory
t3 → Memory
t4 → Memory
t5 → Memory

Instruction 1: t1 = a + b

LOAD R0, a
ADD R0, b

Register Descriptor:

R0 = t1
R1 = Empty

Address Descriptor:

t1 → R0, Memory

Instruction 2: t2 = c * d

LOAD R1, c
MUL R1, d

Register Descriptor:

R0 = t1
R1 = t2

Address Descriptor:

t1 → R0, Memory
t2 → R1, Memory

Instruction 3: t3 = t1 - t2

SUB R0, R1
STORE t3, R0

Register Descriptor:

R0 = t3
R1 = t2

Address Descriptor:

t3 → R0, Memory
t2 → R1, Memory

Instruction 4: t4 = e + f

R1 can be reused because t2 is no longer required.

LOAD R1, e
ADD R1, f

Register Descriptor:

R0 = t3
R1 = t4

Address Descriptor:

t3 → R0, Memory
t4 → R1, Memory

Instruction 5: t5 = t3 / t4

DIV R0, R1
STORE t5, R0

Final Register Descriptor:

R0 = t5
R1 = t4

Final Address Descriptor:

t3 → R0, Memory
t4 → R1, Memory
t5 → R0, Memory

Conclusion:

The two registers are reused efficiently. R0 holds the main computation result, while R1 is reused when the previous temporary value t2 is no longer required.

QUESTION 5

Given Expression:

(a - b) * (c + d) - e

Assume the target machine contains registers R0 and R1.

Step 1: Calculate (a - b)

LOAD R0, a
SUB R0, b

Now:

R0 = a - b

Step 2: Calculate (c + d)

LOAD R1, c
ADD R1, d

Now:

R1 = c + d

Step 3: Multiply the Two Results

MUL R0, R1

Now:

R0 = (a - b) * (c + d)

Step 4: Subtract e

LOAD R1, e
SUB R0, R1

Now:

R0 = (a - b) * (c + d) - e

Step 5: Store the Final Result

STORE result, R0

Complete RISC Target Code:

```
LOAD R0, a
SUB R0, b
LOAD R1, c
ADD R1, d
MUL R0, R1
LOAD R1, e
SUB R0, R1
STORE result, R0
```

Runtime Storage Management:

Runtime storage is used to store input variables, temporary values, local variables, registers, and the final result. Registers are used for frequently accessed intermediate values because register access is faster than memory access.

Stack Frame Layout:

Higher Address

Return Information
Saved Registers
Local Variables
Temporary Storage
Final Result

Lower Address

The stack frame is created when a function begins execution and removed when the function finishes. Local variables and temporary values can be stored in the stack frame when they cannot be kept in registers.

Conclusion:

The RISC target code evaluates the expression step by step using LOAD, ADD, SUB, MUL, and STORE instructions. Register reuse reduces unnecessary memory accesses and improves execution efficiency.

ADDITIONAL OPTIMIZATION PROBLEM

Given Three-Address Code:

```
t1 = a + b
t2 = t1 + 0
t3 = t2 * 1
if t3 == 0 goto L1
t4 = t3 + t3
```

Optimization:

Step 1: Algebraic Simplification

t2 = t1 + 0

Using x + 0 = x:

t2 = t1

Step 2: Algebraic Simplification

t3 = t2 * 1

Using x * 1 = x:

t3 = t2

Step 3: Copy Propagation

Since:

t2 = t1
t3 = t2

we can directly use t1:

t3 = t1

Therefore, the optimized code becomes:

t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1

Final Optimized Code:

t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1

Conclusion:

The optimization removes the unnecessary operations +0 and *1 and eliminates redundant temporary variables. The optimized code requires fewer instructions and improves execution efficiency.